The Essential of Chapter 2

Storage in four pages: pages as the unit of cost, rows per page, heap / sorted / hashed, rows versus columns, what `VACUUM` really does

Advanced Data Management

2026-10-31

Numbers to carry: page 8 KB, page header 24 B, slot 4 B, tuple header 24 B aligned to 8; sequential 500 MB/s (16 μs per page), random SSD 100 μs, random HDD 10 ms; `orders` 500,000 rows / 3,185 pages (157 per page), `customers` 73 pages, `events` 50,000,000 rows / 1,000,000 pages (50 per page).

1 The page is the unit of cost

Concept: Cost = pages touched

A device reads whole blocks; the DBMS reads whole 8 KB **pages**. To get one row you read its page. One I/O costs 10^4–10^6 instructions (SSD tens of μs, HDD ~10 ms, CPU ~1 ns), so for anything touching disk the I/O term dwarfs the CPU term. The cost of an operation = pages read and written; a full scan of a B-page table costs B. Why pages: within a few percent of the truth, hardware-independent, computable by hand from catalog statistics (B, $|R|$, M); the planner encodes the same judgement (`seq_page_cost` 1.0 against `cpu_tuple_cost` 0.01).

Pages to seconds: $\times 16$ μs sequential, $\times 100$ μs random SSD, $\times 10$ ms random HDD: `orders` 51 ms / 0.32 s / 32 s; `events` 16 s / 100 s / 2.8 h.

When the model lies: CPU dominates. Heavy work per page on data already in memory: a sort or hash aggregate over a cached table, `LIKE '%x%'` or a regex on wide text, `jsonb` extraction, an expensive function per row. You notice it in `EXPLAIN (ANALYZE, BUFFERS)`: buffers almost all `shared hit`, `read` near zero, yet a large time in a node doing no I/O; run it twice, a page-bound query gets much faster warm, a CPU-bound one does not.

Mock drill

Mock 1, 2.2: why pages (three reasons), one CPU-bound situation, how to notice it. **Mock 1, 3.2(a)**: 50M/50 = 1,000,000 pages = 8 GB = 16 s cold, the floor for any plan reading `events`. **Mock v2 A7(c)**: `hit=40`

`read=3145` sums to 3,185, all of `orders`.

2 Inside a page: slots and rows per page

Mechanism: Slotted page

Header (24 B: free-space pointers, #slots); a **slot directory** growing down from the header, 4 B per slot holding (offset, length) of a tuple; tuples growing up from the bottom; free space in between. A row is identified by **(page, slot)** = `ctid`, never by byte offset. Hence: tuples can be moved inside the page (compaction) without touching any index; a delete just frees a slot; variable-length rows coexist. A value too big for a page is compressed and stored out of line (**TOAST**), leaving an 18-byte pointer.

Mechanism: Rows per page, pages per table

$$\text{record} = \text{columns} + 24 \text{ (header)}, \text{ rounded up to a multiple of } 8$$

$$\text{rows/page} = \left\lfloor \frac{8{,}192 - 24}{\text{record} + 4} \right\rfloor, \qquad \text{pages} = \left\lceil \frac{\text{rows}}{\text{rows/page}} \right\rceil.$$

Size = pages $\times 8$ KB; scan cost = pages. The floor is not optional: records do not straddle pages.

Worked: 150-byte records: $\lfloor 8{,}168/154 \rfloor = \lfloor 53.04 \rfloor = 53$ per page (54 forgets the slot, 53.04 forgets the floor). **Quiz C1, `tickets`:** columns $8 + 4 + 12 + 8 + 8 = 40$, $+24 = 64$ (aligned), $+4$ slot $= 68$; $\lfloor 8{,}168/68 \rfloor = 120$ rows/page; $\lceil 3\text{M}/120 \rceil = 25{,}000$ pages $= 200$ MB, 0.4 s cold. Add a 100-byte `notes` column: $140 + 24 = 164 \to 168$, $+4 = 172$; 47 rows/page; 63,830 pages; 511 MB; every scan 2.55× slower even for queries that never read `notes`.

Mock drill

Quiz chapter 2 block: the formula with header 24 and slot 4; why offsets in the slot directory (compaction without changing the ids indexes store). **Quiz C1**: tuple size with header and alignment, rows per page, pages, MB, then again after the new column, floor and ceiling shown.

3 Heap, sorted and hashed files

Operation (B pages)	Heap	Sorted	Hashed
Full scan	B	B	B
Equality, key present	$B/2$ avg	$\lceil \log_2 B \rceil$	~ 1
Equality, key **absent**	B (must read all)	$\lceil \log_2 B \rceil$	~ 1
Range	B	$\log_2 B+$ matching pages	B (order destroyed)
Insert	~ 1	up to B (shift in key order)	~ 1

No single winner, so real systems store a **heap plus indexes** (ch. 3): log or ~ 1 lookups without the sorted file's insert price. PostgreSQL tables are heaps.

Mock drill

Quiz quick checks: an absent key costs B, not $B/2$; a sorted insert costs up to B. **Quiz C2**: exact-match lookups and batch loads $\rightarrow$ hashed; append-only log read by scans $\rightarrow$ heap; "all rows between two dates" $\rightarrow$ sorted by date; mixed workload $\rightarrow$ heap + B$^+$ tree ($h + 1 \approx 4$ reads per lookup, cheap inserts).

4 Rows versus columns

Row store (NSM): a page holds whole rows; one record is one page read and one write: right for OLTP. **Column store** (DSM): a page holds a slice of one column; `avg(amount)` on a 20-column table reads 1/20 of the data, and same-type runs compress (run-length, dictionary, delta) far better than mixed rows: right for OLAP. The price: a single-row update touches 20 pages instead of 1.

Mock drill

Mock 1, 3.2(b): claim (one column is read), number (8 GB row scan against 50M $\times$ 8 = 400 MB raw against tens of MB delta-encoded, 50–100$\times$ fewer bytes), mechanism (page per column, same-type compression, min/max row-group skipping); then the `COPY ... (FORMAT parquet)` export. **Quiz**: 1/30 of the pages for one of 30 columns. Never write "column stores are faster".

5 PostgreSQL storage and `VACUUM`

Files of up to 1 GB, 8 KB pages, heap, slotted, row store; the tuple header holds `t_xmin`/`t_xmax`; the **free-space map** records the room per page for inserts. `heap_page_items` shows `lp_off` decreasing with the slot number (8152, 8112): tuples really grow up from the bottom.

```
SELECT pg_relation_size('orders') / 8192 AS pages;                -- 3185
SELECT count(*) FROM heap_page_items(get_raw_page('orders', 0));  -- ~157
```

Concept: Dead tuples, `VACUUM`, `VACUUM FULL`

`DELETE` only stamps `t_xmax`; the tuple becomes **dead** but keeps its bytes. Plain `VACUUM` removes dead tuples, compacts *inside* each page (slots updated, `ctid`s unchanged), records the room in the FSM, and truncates only *trailing pages that became completely empty*. It never moves a live row to another page, so the file length does not change unless the last pages are empty. `VACUUM FULL` / `CLUSTER` rewrite the table into a new, dense file (exclusive lock, full copy).

Mock v2 A2: delete half of `orders` at random, plain `VACUUM`. Every page keeps about 78 live rows, none empties, nothing to truncate: **about 3,185 pages** (b), each half free; the next 250,000 inserts fill the holes. `VACUUM FULL` would give $\lceil 250{,}000/157 \rceil \approx 1{,}592$ (a, the trap).

Mock drill

Mock v2 A2: (b) with "VACUUM frees space inside pages and truncates only empty trailing pages; random deletes empty no page"; name `VACUUM FULL` for 1,592. **Mock 1, 3.2(c)**: `pg_relation_size/8192` (1,000,000) and `count(*)` over `heap_page_items` of page 0 (≈ 50); the `jsonb` payload lives in TOAST so it does not distort the count.

Warning / Gotcha: The four mistakes that cost points

- Forgetting the slot or the floor: record +4; 78.5 rows is 78; add the 24-byte header and align to 8 first.
- "An absent key costs $B/2$." Proving absence reads every page: B.
- "`VACUUM` shrinks the file." Only `VACUUM FULL` / `CLUSTER` rewrite; random deletes leave the page count unchanged.
- "Column stores are simply better." 20 pages for one row update; the workload decides.

Summary: The essential in five lines

1. Cost = pages; one I/O = 10^4–10^6 instructions. Lies only for heavy CPU on cached data (hit high, read ≈ 0, no speed-up warm).
2. Slotted page: stable (page, slot) ids. Rows/page = $\lfloor 8{,}168/(\text{record} + 4) \rfloor$, record = columns +24 aligned to 8; `orders` 157 per page.
3. Heap $B/2$ present, B absent, 1 insert; sorted $\log_2 B$, ranges, insert up to B; hashed ~ 1, no ranges. Real systems: heap + indexes.
4. Row store for OLTP; column store for OLAP (1/20 of the I/O for one of 20 columns; 20 pages per row update).
5. `VACUUM` frees inside pages, truncates only empty trailing pages: 3,185 stays 3,185; `VACUUM FULL` gives 1,592.